Solucionario Java — Actividad de Aprendizaje Semana 01

Curso: Programación Orientada a Objetos II — Módulo 1

Este documento desarrolla en Java las dos actividades de la guía "Actividad de Aprendizaje - Semana 01": la clase Círculo (encapsulamiento, constructores sobrecargados y this) y la jerarquía de Figuras Geométricas (herencia, clases abstractas, composición y polimorfismo). Cada actividad incluye el código fuente completo, listo para compilar y ejecutar.

Índice

Actividad 1 — Clase Círculo

- 1.1 Clase Circulo.java
- 1.2 Clase CirculoTest.java
- 1.3 Diagrama de clases (PlantUML)

Actividad 2 — Jerarquía de Figuras Geométricas

- 2.1 Clase Punto.java
- 2.2 Clase FiguraGeometrica.java (abstracta)
- 2.3 Clase Triangulo.java
- 2.4 Clase SuperficiePlana.java
- 2.5 Clase Main.java

Actividad 1 — Clase Círculo

Se modela un Círculo con dos atributos privados (radio y color), dos constructores sobrecargados y los métodos getRadio() y getArea(). Luego, CirculoTest ejercita ambos constructores, demuestra el uso de this y de setters, y agrega toString() para representar el objeto como texto.

1.1 Clase Circulo.java

radio y color son `private`: solo son accesibles desde dentro de la propia clase. El constructor con (radio, color) usa `this` para distinguir el atributo de instancia del parámetro que tiene el mismo nombre, y delega en setters para validar/asignar los valores.

`src/main/java/poo2/actividad1/Circulo.java`

```
package poo2.actividad1;

public class Circulo {

    private double radio;
```

```
    private String color;

    public Circulo() {
        this(12.5, "azul");
    }

    public Circulo(double radio) {
        this(radio, "azul");
    }

    public Circulo(double radio, String color) {
        this.setRadio(radio);
        this.setColor(color);
    }

    public double getRadio() {
        return radio;
    }

    public void setRadio(double radio) {
        this.radio = radio;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

    public double getArea() {
        return radio * radio * Math.PI;
    }

    @Override
    public String toString() {
        return "Circulo{radio=" + radio + ", color='" + color + "', area=" + getArea() + "}";
    }
}
```

1.2 Clase CirculoTest.java

Contiene el método `main()` que imprime radio y área usando el constructor por defecto y el sobrecargado (radio = 6.55), crea un objeto y modifica sus valores por defecto con los setters, e imprime el resultado con toString().

`src/main/java/poo2/actividad1/CirculoTest.java`

```
package poo2.actividad1;

public class CirculoTest {

    public static void main(String[] args) {

        System.out.println("== Constructor por defecto ==");
        Circulo c1 = new Circulo();
        System.out.println("Radio: " + c1.getRadio());
```

```
        System.out.println("Area: " + c1.getArea());

        System.out.println();
        System.out.println("== Constructor sobrecargado (radio = 6.55) ==");
        Circulo c2 = new Circulo(6.55);
        System.out.println("Radio: " + c2.getRadio());
        System.out.println("Area: " + c2.getArea());

        System.out.println();
        System.out.println("== Objeto modificado con setters ==");
        Circulo c3 = new Circulo();
        c3.setRadio(20.0);
        c3.setColor("rojo");
        System.out.println(c3);

        System.out.println();
        System.out.println("== Constructor con this (radio, color) ==");
        Circulo c4 = new Circulo(8.0, "verde");
        System.out.println(c4);
    }
}
```

1.3 Diagrama de clases (PlantUML)

Diagrama de la clase Circulo para pegar en `https://plantuml.com/es/class-diagram` o en la extensión de PlantUML del IDE.

`diagrama-circulo.puml`

```
@startuml
class Circulo {
  - radio: double
  - color: String
  + Circulo()
  + Circulo(radio: double)
  + Circulo(radio: double, color: String)
  + getRadio(): double
  + setRadio(radio: double): void
  + getColor(): String
  + setColor(color: String): void
  + getArea(): double
  + toString(): String
}
@enduml
```

Volver al índice · Siguiente: Actividad 2 — Jerarquía de Figuras Geométricas

Actividad 2 — Jerarquía de Figuras Geométricas

Se modela una jerarquía con Punto, la clase abstracta FiguraGeometrica (con los métodos abstractos calcularArea() y esRegular()), Triangulo como subclase concreta, y SuperficiePlana, que compone una

lista de figuras y devuelve sus áreas. Main.java es la clase genérica que instancia y prueba todo el conjunto.

2.1 Clase Punto.java

toString() devuelve exactamente el formato pedido en el enunciado. calcularDistancia() está sobrecargado: la versión sin parámetros delega en la versión con parámetro usando el punto (0,0) como referencia.

`src/main/java/poo2/actividad2/Punto.java`

```
package poo2.actividad2;

public class Punto {

    private double x;
    private double y;

    public Punto() {
        this(0.0, 0.0);
    }

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public void setX(double x) {
        this.x = x;
    }

    public double getY() {
        return y;
    }

    public void setY(double y) {
        this.y = y;
    }

    public double calcularDistancia() {
        return calcularDistancia(new Punto(0.0, 0.0));
    }

    public double calcularDistancia(Punto otro) {
        double dx = this.x - otro.x;
        double dy = this.y - otro.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    @Override
    public String toString() {
        return "El punto tiene las siguientes coordenadas: " + x + "," + y;
    }
```

```
}
```

2.2 Clase FiguraGeometrica.java (abstracta)

Los atributos son `protected`: visibles en la propia clase, en el mismo paquete y en subclases. calcularArea() y esRegular() son abstractos: toda subclase concreta debe implementarlos.

`src/main/java/poo2/actividad2/FiguraGeometrica.java`

```
package poo2.actividad2;

public abstract class FiguraGeometrica {

    protected String nombre;
    protected Punto[] vertices;

    public FiguraGeometrica(String nombre, Punto[] vertices) {
        this.nombre = nombre;
        this.vertices = vertices;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Punto[] getVertices() {
        return vertices;
    }

    public void setVertices(Punto[] vertices) {
        this.vertices = vertices;
    }

    public abstract double calcularArea();

    public abstract boolean esRegular();

    @Override
    public String toString() {
        return nombre + " -> area=" + calcularArea() + ", regular=" + esRegular();
    }
}
```

2.3 Clase Triangulo.java

Triangulo extiende FiguraGeometrica (herencia) y se compone de 3 objetos Punto (composición). calcularArea() usa la fórmula del determinante (shoelace) y esRegular() compara los tres lados con calcularDistancia().

`src/main/java/poo2/actividad2/Triangulo.java`

```
package poo2.actividad2;

public class Triangulo extends FiguraGeometrica {

    public Triangulo(Punto p1, Punto p2, Punto p3) {
        super("Triangulo", new Punto[] { p1, p2, p3 });
    }

    @Override
    public double calcularArea() {
        Punto a = vertices[0];
        Punto b = vertices[1];
        Punto c = vertices[2];
        return Math.abs(
                (a.getX() * (b.getY() - c.getY()))
              + (b.getX() * (c.getY() - a.getY()))
              + (c.getX() * (a.getY() - b.getY()))
        ) / 2.0;
    }

    @Override
    public boolean esRegular() {
        double ladoAB = vertices[0].calcularDistancia(vertices[1]);
        double ladoBC = vertices[1].calcularDistancia(vertices[2]);
        double ladoCA = vertices[2].calcularDistancia(vertices[0]);
        return ladoAB == ladoBC && ladoBC == ladoCA;
    }
}
```

2.4 Clase SuperficiePlana.java

Composición: SuperficiePlana "tiene" una lista de FiguraGeometrica y expone obtenerAreas(), que recorre la lista y devuelve el área de cada figura aplicando polimorfismo (cada figura calcula su área con su propia implementación).

src/main/java/poo2/actividad2/SuperficiePlana.java

```
package poo2.actividad2;

import java.util.ArrayList;
import java.util.List;

public class SuperficiePlana {

    private List<FiguraGeometrica> figuras = new ArrayList<>();

    public void agregarFigura(FiguraGeometrica figura) {
        figuras.add(figura);
    }

    public List<FiguraGeometrica> getFiguras() {
        return figuras;
    }

    public List<Double> obtenerAreas() {
        List<Double> areas = new ArrayList<>();
        for (FiguraGeometrica figura : figuras) {
            areas.add(figura.calcularArea());
```

```
        }
        return areas;
    }
}
```

2.5 Clase Main.java

Clase genérica que instancia Puntos, arma un Triangulo, lo agrega a una SuperficiePlana y muestra el resultado. La línea final asigna el Triangulo a una referencia de tipo FiguraGeometrica para evidenciar el polimorfismo.

`src/main/java/poo2/actividad2/Main.java`

```
package poo2.actividad2;

public class Main {

    public static void main(String[] args) {

        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(4, 0);
        Punto p3 = new Punto(0, 3);

        Triangulo triangulo = new Triangulo(p1, p2, p3);

        SuperficiePlana superficie = new SuperficiePlana();
        superficie.agregarFigura(triangulo);

        System.out.println(triangulo);
        System.out.println("Areas registradas: " + superficie.obtenerAreas());

        // Polimorfismo: la referencia es del tipo de la superclase,
        // pero en tiempo de ejecucion se invoca la implementacion de Triangulo.
        FiguraGeometrica figura = triangulo;
        System.out.println("Area via referencia polimorfica: " + figura.calcularArea());
        System.out.println("Es regular: " + figura.esRegular());
    }
}
```

Volver al índice